

SO2 — Etapa 2

Comunicação entre componentes de um mesmo Sistema Autônomo

Integrantes: Arthur Erpen, Caetano Peruzzo e Gabriel Herrera

UFSC — Sistemas Operacionais II

Abril de 2026

Agenda

- 1 Contexto e Objetivo
- 2 Arquitetura Geral
- 3 Identificadores
- 4 A Região de Memória Compartilhada
- 5 Fluxo de Envio e Recepção
- 6 Gateway e Roteamento
- 7 Mensagem, RT e Biblioteca
- 8 Testes
- 9 Encerramento

Contexto e requisitos da Etapa 2

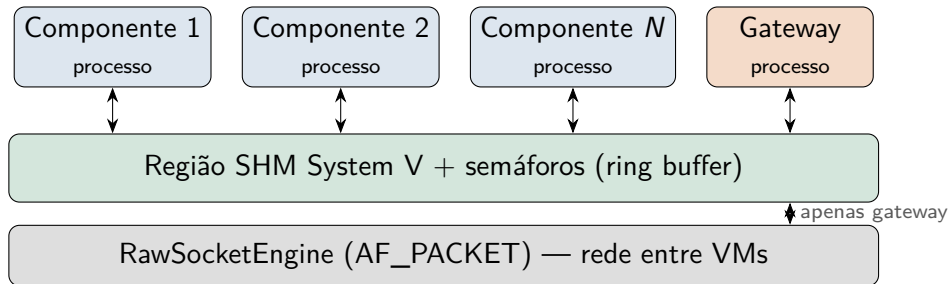
O que já tínhamos (Etapa 1)

- API Communicator / Protocol / NIC / Engine.
- RawSocketEngine sobre AF_PACKET.
- Cada VM era *um único processo*.
- Comunicação entre VMs.

O que muda agora

- Cada VM vira **gateway + N componentes**, cada um em seu processo.
- Segunda Engine sobre **SHM System V** (shmget + semget), produtor $\times$ consumidor.
- Toda a diferença fica confinada à Engine.
- Mensagem ganha Origin:
 $M = \{address, payload\}$.
- Propagação imediata, sem filas intermediárias.

Visão de alto nível de uma VM



Componentes só enxergam a SHM. **Só o gateway** possui ambas as NICs e faz a ponte entre a SHM local e a rede.

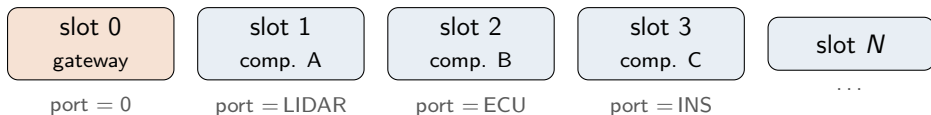
Duas engines, um mesmo Protocol

```
template <typename SharedMemoryNIC, typename RawSocketNIC = void>
class Protocol
    : private SharedMemoryNIC::Observer,
      public Concurrent_Observed<Buffer<Ethernet::Frame>, uint16_t>
{
    SharedMemoryNIC    _shm_nic;        // sempre presente
    RawSocketNIC *     _socket_nic;    // apenas no gateway
    // ...
};
```

- Protocol é parametrizado por duas NICs.
- `_socket_nic` só é instanciado se `SharedMemoryNIC::is_gateway_process()`.
- Componentes comuns nem alocam recursos de rede — só SHM.

Slot, Port e por que não usar PID

`Protocol::Address` = {`Physical_Address`, `Port`}. Usar PID como `Port` seria barato e único localmente, mas **uma VM remota não tem como descobrir o PID de um processo de outra VM**. Por isso separamos dois conceitos:



- **slot**: índice fixo do processo na VM (slot 0 reservado ao gateway). Endereça semáforos e resolve *self-drop* na recepção.
- **port**: porta lógica do componente (tipo), vai no cabeçalho do `Protocol::Packet` e é globalmente descobrível.

Layout de SHM::Region

<code>uint32_t magic = 0x534F3232 ("SO22")</code>
<code>uint16_t component_count uint8_t gateway_active</code>
<code>bootstrap_ready_count / bootstrap_released</code>
<code>Component_Entry components[MAX_COMPONENTS]</code> <code>{port, slot, active, receiver_ready}</code>
<code>Broadcast_Ring ring</code> <code>next_write_seq + Broadcast_Slot slots[SLOT_COUNT]</code>

Cada Broadcast_Slot carrega seq, writer_slot, remaining_readers, flags e o frame Ethernet inteiro.

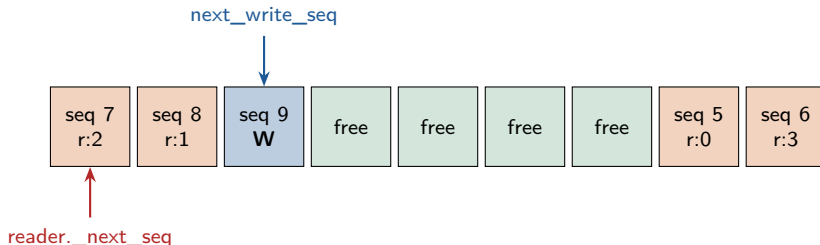
Semáforos System V

Um único semid com numeração fixa:

```
enum Semaphore_Index {  
    SEM_RING_MUTEX           = 0, // mutex do ring  
    SEM_FREE_SLOTS           = 1, // slots livres (inicia em SLOT_COUNT)  
    SEM_GATEWAY_PENDING      = 2, // msgs pendentes p/ gateway  
    SEM_BOOTSTRAP_MUTEX      = 3, // barreira de startup  
    SEM_BOOTSTRAP_RELEASE    = 4, // libera run() simultaneamente  
    SEM_COMPONENT_PENDING_BASE = 5 // +1 por componente (pending_i)  
};
```

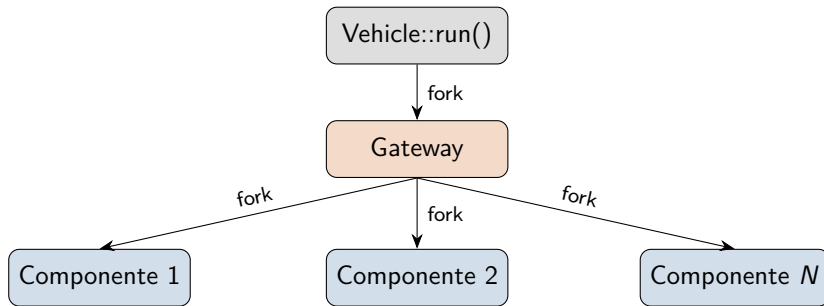
- FREE_SLOTS inicia em SLOT_COUNT (100) — produtor/consumidor clássico.
- Cada componente ativo tem *seu próprio* pending_i: o writer dá V() em todos os leitores.

Ring buffer — produtor × consumidor



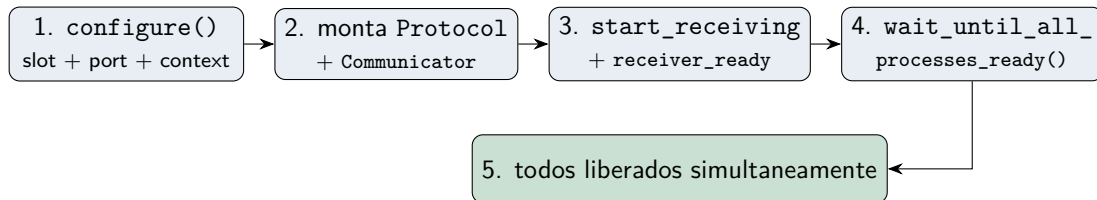
- **Writer:** $P(\text{FREE}) \rightarrow P(\text{MUTEX}) \rightarrow \text{escreve slot} \rightarrow \text{seta remaining_readers} \rightarrow V(\text{pending}_i) \forall$ leitor ativo.
- **Reader:** $P(\text{pending}_{\text{self}}) \rightarrow P(\text{MUTEX}) \rightarrow \text{copia slots}[\text{next_seq} \% N] \rightarrow \text{decrementa}; \text{ se zerou, } V(\text{FREE}).$
- Flags no slot decidem destino (DELIVER_TO_COMPONENTS, DELIVER_TO_GATEWAY, FROM_NETWORK) e writer_slot garante o self-drop.

Quem cria a infraestrutura?



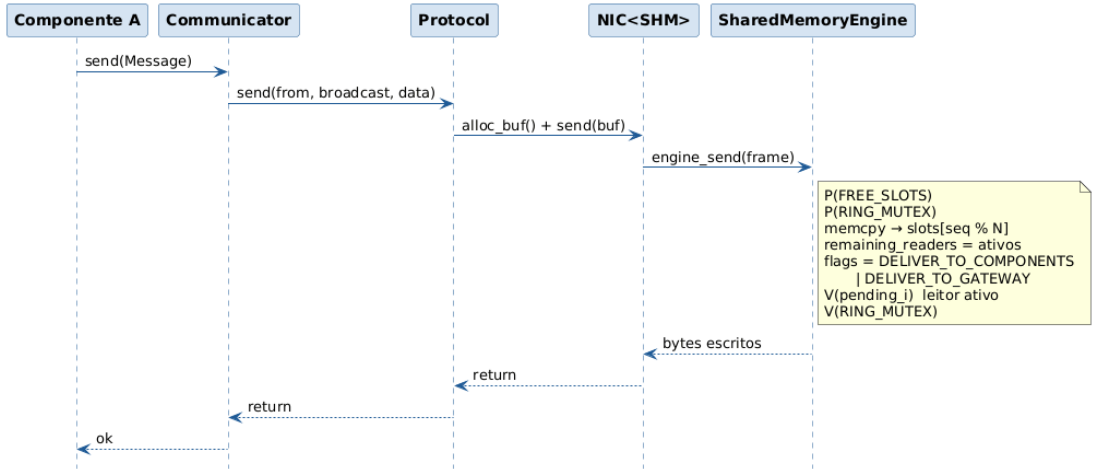
Gateway é o pai: `shmget/semget`, depois `fork()` de cada componente, e só então roda o próprio loop. Destruição (`IPC_RMID`) acontece no mesmo processo ao sair.

Barreira de bootstrap

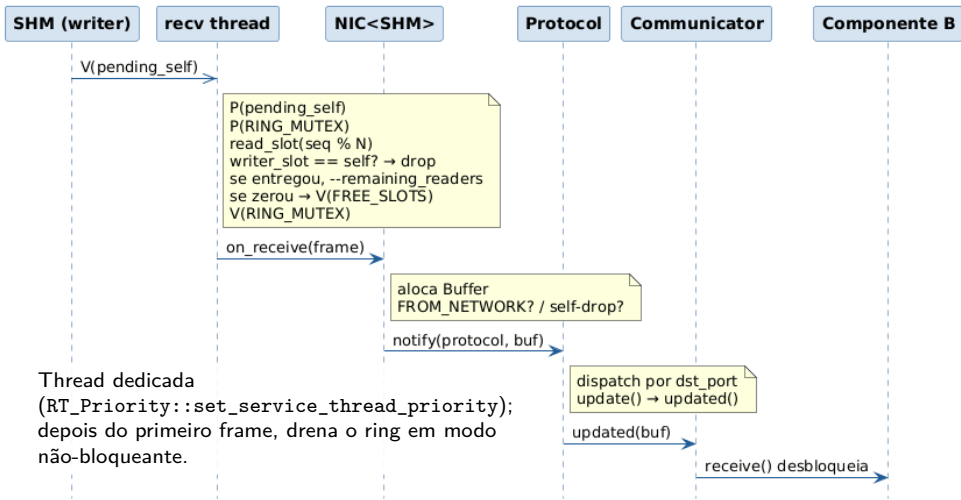


- Usa SEM_BOOTSTRAP_MUTEX + SEM_BOOTSTRAP_RELEASE.
- Nenhum Component::run() começa antes que *todos* os receivers tenham marcado receiver_ready = 1.
- Fecha a janela em que mensagens escritas “cedo demais” sumiriam sem leitor.

Sequência — envio intra-VM

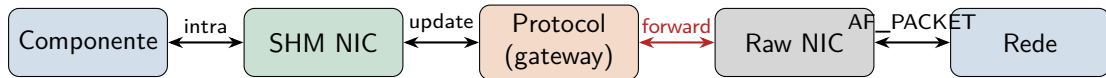


Sequência — recepção intra-VM



Gateway faz a ponte no Protocol::update()

- Frame chega pela **SHM**, com `src == MAC do gateway` e `dst == BROADCAST`: copia para um buffer da `RawSocketNIC` e reenvia para a rede.
- Frame chega pela **rede**: aloca buffer na SHM e injeta o mesmo Ethernet frame.
- Caso contrário: entrega local por `dst_port`.
- **Propagação imediata**: o forward acontece na mesma observação que dispararia o `update()` — sem fila nem thread intermediária.



Mensagem com Origin

```
class Message {
public:
    struct Origin {
        Ethernet::Address address;
        uint16_t port;
    };
    const Origin & origin() const;
    void origin(const Origin & o);
    // data() / size() ...
};
```

- `Communicator::receive()` preenche `message.origin({from.paddr(), from.port()})`.
- O par **(MAC, porta lógica)** identifica unicamente o emissor mesmo entre VMs — base para a resposta na Etapa 3.

Prioridade de tempo real + biblioteca estática

RT priority (src/core/rt_priority.[h|cpp]):

- SCHED_FIFO, prioridade 98 (main) e 99 (thread de recepção SHM).
- Aplicado no run_component_process, run_gateway_process e na service thread — a recepção nunca fica atrás de um componente “ocupado”.

Biblioteca estática build/lib/libso2.a:

```
$(STATIC_LIB): $(OBJS)
    $(AR) rcs $@ $^

$(BIN_DIR)/%: $(TEST_DIR)/%.cpp $(STATIC_LIB)
    $(CXX) $(CXXFLAGS) $< -L$(LIB_DIR) -l$(LIB_NAME) -o $@
```

- src/ vira um .a único; testes em tests/ são clientes comuns (#include + -lso2).
- Cross-compilado para riscv64-linux-gnu e rodado em QEMU.

Testes em destaque: stress e rtt_intra

Suite automatizada em QEMU via `makefile + run_qemu_test.sh` (basic, local_broadcast, gateway_path, v3, stress, rtt, rtt_intra). Destaque para os dois que *efetivamente estressam* a Etapa 2:

stress — carga intra + inter VM

- 5 VMs, múltiplos componentes cada, centenas de mensagens por sender.
- Detecta vazamento de slot ou `pendingi` dessincronizado (travaria).
- Prova ausência de deadlock entre `FREE_SLOTS`, `MUTEX` e `pendingi`, com forward do gateway em paralelo ao tráfego local.

rtt_intra — latência pura da Engine SHM

- Ping-pong entre dois componentes da *mesma* VM, `CLOCK_MONOTONIC_RAW`, 20 amostras, min/média/p95/max via `summarize_rtt.py`.
- Isola o custo da engine SHM sem `AF_PACKET` nem switch QEMU — mostra que o caminho intra é dominado por `semop + memcpy`.

Resultados do stress — perda de pacotes

VM	Intra recv.	Intra perd.	Inter recv.	Inter perd.	Duplicatas
vm1	200 / 200	0 (%)	800 / 800	0 (%)	0
vm2	200 / 200	0 (%)	800 / 800	0 (%)	0
vm3	200 / 200	0 (%)	800 / 800	0 (%)	0
vm4	200 / 200	0 (%)	800 / 800	0 (%)	0
vm5	200 / 200	0 (%)	800 / 800	0 (%)	0
Total	1000 / 1000	0 (%)	4000 / 4000	0 (%)	0

Observação: Na maioria das execuções do teste obtivemos esses resultados, entretanto, a depender da máquina que foi executado em algumas execuções obtivemos perdas de até 3% de pacotes.

Dados obtidos ao executar make.

Resultados do rtt_intra — latência

Teste	Amostras	Min	Média	Max
rtt_intra	20	3381 us	8454.10 us	46012 us

Dados obtidos ao executar `make measure-rtt-intra`.

O que entregamos nesta etapa

- SharedMemoryEngine completa sobre shmget/semget, encaixada no mesmo molde de Engine da Etapa 1.
- Protocol parametrizado por *duas* NICs, com roteamento local/remoto no próprio update() e propagação imediata.
- Gateway como processo pai: cria/destrói a SHM e faz ponte com a rede.
- Bootstrap barrier evitando corridas na primeira mensagem.
- Message carregando Origin (MAC + porta).
- SCHED_FIFO nas threads críticas + empacotamento como libso2.a.
- Suite automatizada em QEMU, com stress e rtt_intra como evidência principal.

Obrigado!